Exploring Solidity — Lecture Notes

1. Introduction to Solidity

a) **Solidity** is a **programming language** for writing **Ethereum smart contracts**.
b) It's the **most popular** and widely used language for Ethereum.
c) **Syntax style:** similar to **JavaScript**.
    a. Easy to learn, especially for those with JavaScript background.
d) **Purpose:** Used to create logic that runs on **Ethereum Virtual Machine (EVM)** nodes.

2. Key Characteristics

a) **Runs on all EVMs / nodes** → execution must be **deterministic**.
    a. Same input → same output everywhere.
    b. Avoid using **external data** that may differ across nodes.
b) Smart contracts act as **gatekeepers** to the blockchain — you can't access blockchain data directly without them.
c) **Code immutability:** Once deployed, code is **stored permanently** on the blockchain.
    a. Must be **fully tested** before deployment.

3. Compilation Process

a) Developers write **human-readable source code**.
b) Compiler translates it into **bytecode (opcodes)** → executed by EVM.
c) Like any programming language:
    a. Must follow **syntax** and **semantic** rules.

4. Syntax vs. Semantics

Aspect	Description
Syntax	Rules for valid structure (grammar, symbols, keywords). Errors here prevent compilation.
Semantics	Meaning and logic of the program. Even syntactically correct code can behave wrongly.
Tip	Always ensure both syntactical correctness and correct logic via testing.

---

5. Main Elements of a Solidity File

A Solidity contract typically includes:

1. **Compiler version declaration**
2. **Comments**
3. **Import statements (optional)**
4. **Contract definition**

a) Compiler Version Declaration

b) Solidity evolves quickly → newer versions may **break compatibility**.

c) Use **pragma directive** to specify compiler version.

d) `pragma solidity ^0.5.0;`

   a. `^` (caret) means **compatible with 0.5.x versions only**.

   b. To define a range:

   c. `pragma solidity >=0.5.0 <0.6.0;`

e) Prevents compiling with incompatible versions.

7. Comments in Solidity

Used for code readability, team collaboration, and self-documentation.

Type	Syntax	Example
Single-line	`//`	`// This is a demo smart contract`
Inline	After code	`uint x; // store balance`
Multi-line	`/* ... */`	`/* This is a long comment */`

Warnings:

1. Yellow underline → Warning (e.g., trailing spaces).
2. Red underline → Error.

8. Importing Other Files

1. Used to include external code (functions, libraries, or variables).
2. Prevents **duplication** and promotes **modularity**.
3. Example (not shown in HelloWorld contract):
4. `import "./MyLibrary.sol";`

9. Contract Definition

1. Defined using the **`contract`** keyword.

```
2. contract HelloWorld {
3.     // contract body
4. }
```

5. The file is typically named **HelloWorld.sol**.
6. The **contract name** should ideally match the **file name** (for clarity).
7. Body enclosed between `{ }` defines all state variables and functions.

10. HelloWorld Example — Structure Overview

```
pragma solidity ^0.5.0;          // Compiler version
// This is a demo smart contract  // Comment

contract HelloWorld {            // Contract definition
    string public message = "Hello World";
}
```

- Simple structure includes:

    1. **Version declaration**
    2. **Comments**
    3. **Contract declaration with body**

11. Conclusion

1) Solidity enables creation of decentralized applications (dApps).
2) Focus on:
    a. Deterministic logic
    b. Proper syntax & semantics
    c. Compiler version management
    d. Clear comments and modular design
3) The **HelloWorld** example is a foundation — more complex features (imports, inheritance, modifiers, etc.) build upon it.

Defining Types of Data — Lecture Notes

1. Understanding Data in Solidity

1) **Data** = any **value we manipulate or store** within a smart contract.
2) Think of it in **linguistic terms:**
3) **Nouns → Data** (values, entities, information)
4) **Verbs → Functions** (actions performed on data)
5) Examples of data:
6) Current token balance
7) Product price
8) Owner's address

2. Blockchain vs Traditional Data Handling

Feature	Traditional Systems (DB/Files)	Blockchain (Solidity)
Storage	Data stored in place (DB, CSV, Excel, etc.)	Data stored in **blocks**
Update	Directly overwrite old data	**No direct update** – add a **new block/version**
Delete	Can delete data	**No deletion** – immutable once stored
Cost	Free/local	Updating = **new transaction** → costs **gas**
Effect	Constant size	Blockchain size grows with each update

Key takeaway: Updating data = *adding data*, not *replacing* it.

3. Two Main Types of Variables

Solidity classifies data mainly by **scope and persistence**:

A. Local Variables

1) Exist **temporarily in memory** during contract execution.
2) Lost when the function call ends.
3) Not stored on blockchain → **no persistence**.
4) **No gas cost** for storage (only for execution).

B. State Variables

1) **Stored permanently** on the blockchain.
2) Retain values between transactions.
3) **Persistent data** across contract calls.
4) Require **gas** to store or modify (paid by user).

Type	Persistence	Storage Location	Gas Cost	Example
Local	Temporary	EVM memory	Low	`uint tempSum = 0;`
State	Permanent	Blockchain	High	`uint totalSupply;`

4. Physical Data Locations in Solidity

Solidity defines **where** data lives in the EVM or blockchain.

Data Location	Description	Usage	Cost
Stack	For **simple types** (e.g., `int`, `bool`, `address`)	Temporary, inside function	Free (local)
Memory	For **complex types** (arrays, structs, strings)	Temporary, for intermediate calculations	Free (local)
Storage	Permanent area on **blockchain**	Holds **state variables**	Paid (gas required)

- **Stack and Memory → local variables**
- **Storage → state variables**

Be mindful where your variable lives.
The more persistent it is, the more costly it becomes.

5. Practical Guidelines

- Use **local variables** when data is temporary and doesn't need persistence.
- Use **state variables** only when the value must survive after execution.
- Each write to storage = **new blockchain transaction → gas fees apply**.
- Minimize storage writes to optimize contract efficiency.

6. Summary

Concept	Key Point
Data in Solidity	Represents all values we can read/write.
Immutability	No delete or overwrite — updates create new data.
Local Variables	Temporary, in memory, no cost to store.
State Variables	Permanent, on blockchain, gas cost applies.
Data Locations	Stack (simple), Memory (complex), Storage (permanent).

Data Types in Solidity — Lecture Notes

1. Introduction

1) Like all programming languages, **Solidity supports multiple data types** for storing and manipulating values.
2) Choosing the **right data type** is crucial because:
    a) It defines **how much memory** or **blockchain storage** is used.
    b) It impacts **gas cost** and **data accuracy**.
3) Each data type has **specific strengths and limitations** in handling internal data.

2. Project Context (Setup Recap)

1) Example project created using **Truffle**:
2) `mkdir SecondProject`
3) `cd SecondProject`
4) `truffle init`
5) Folder structure automatically created:
    a) `contracts/` → for Solidity source files
    b) `migrations/` → for deployment scripts
    c) `test/` → for test cases
6) A sample file **DataTypes.sol** is opened in Visual Studio Code.

3. Basic Solidity Code Structure

```
pragma solidity ^0.8.0;

contract DataTypes {
    uint x;
    uint8 smallNumber;
    int signedNumber;
    bool isActive;
}
```

Each variable in Solidity is declared with a **name** and a **data type**.

4. Unsigned Integers (uint)

1) **`uint`** → *Unsigned integer* (only non-negative values, ≥ 0).

2) Commonly used for **counting quantities**, **IDs**, **balances**, etc.

```
3) uint x = 100;  // Same as uint256 by default
```

4) **Bit variants:** `uint8`, `uint16`, … up to `uint256`
   a) Default `uint` = **`uint256`**
   b) Example:

```
   c) uint8 smallNumber = 25;   // Range: 0 - 255
   d) uint256 bigNumber = 1000; // Range: 0 - 2^256 - 1
```

5) **Optimization Tip:**
   a) Choose the smallest bit size possible to **save gas** (smaller data = cheaper storage).
   b) But if you exceed the range, the program throws an **overflow error**.

Type	Range	Size	Typical Use
`uint8`	0 – 255	1 byte	small counters, flags
`uint16`	0 – 65,535	2 bytes	medium values
`uint256`	0 – 2^{256}-1	32 bytes	balances, big numbers

5. Signed Integers (int)

1) **`int`** → signed integer (can hold **negative and positive** numbers).
   a) Used when you need values that may go below zero.

```
2) int temperature = -5;
3) int balanceChange = 120;
```

4) Like `uint`, you can define **bit length variants**:
   `int8`, `int16`, `int32`, … up to `int256`
5) **Default:** `int` = `int256`.

Type	Range	Example Use
`int8`	–128 to 127	small counters
`int256`	-2^{255} to 2^{255}-1	financial or signed calculations

6. Boolean (bool)

1) **`bool`** stores **true/false** values only.

```
2) bool isActive = true;
3) bool isOwner = false;
```

4) Commonly used for:
   a) **Condition checks** (`if`, `while` statements)
   b) **Flags or permissions** in smart contracts.

Type	Values	Example
`bool`	`true` / `false`	`bool success = true;`

7. Gas Optimization Reminder

1) Larger data types → more bits → more **storage cost** on the blockchain.
2) Example:
    a) `uint256` costs **more gas** than `uint8` if you only need small values.
3) **Best practice:** Use the smallest data type that fits your expected value range.

8. Coding Hygiene (Best Practices)

1) Watch for **VS Code warnings** (yellow lines → minor issues, red lines → syntax errors).
2) Remove **trailing spaces** and keep the code clean for better readability.
3) Comment your code to explain variable roles.

9. Summary Table

Data Type	Description	Range / Values	Example
`uint`	Unsigned integer	0 to 2^{256}-1	`uint count = 5;`
`uint8`	Small unsigned integer	0–255	`uint8 level = 3;`
`int`	Signed integer	-2^{255} to 2^{255}-1	`int temp = -5;`
`bool`	Logical type	`true` / `false`	`bool done = false;`

10. Key Takeaways

1) Solidity's basic numeric types mirror traditional languages but add **gas cost implications**.
2) Prefer **compact types** to save gas, but ensure they fit the value range.
3) `uint` = default for counts and amounts; `int` = when negatives are possible; `bool` = for conditions.
4) Solidity's design balances **type safety**, **efficiency**, and **blockchain immutability**.

Solidity Data Modifiers — Part 1

1. Introduction to Visibility Modifiers

1) **Visibility** in Solidity defines **who can access variables** and **who can invoke functions**.
2) It controls the **scope of accessibility**—inside or outside the smart contract.
3) Every variable (the *data bucket*) and every function (the *action*) can have restricted or open visibility.

Analogy:

1) The *bucket (variable)* holds data.
2) The *hand (function or caller)* decides who can reach into the bucket.

2. Why Visibility Matters

1. Provides **security** and **access control** within contracts.
2. Helps protect internal logic and sensitive data.
3. Ensures that external users, dApps, or APIs can only access **intended parts** of the contract.

3. Four Main Visibility Modifiers

Solidity defines **four primary modifiers** that determine who can interact with a function or variable:

Modifier	Accessible From	Typical Use	Applied To
public	Anywhere (internal + external)	Open functions or variables for global access	Functions & Variables
external	Only from outside the contract	APIs or interfaces for external calls	Functions only
internal	Current contract + derived contracts	Shared logic for inheritance	Functions & Variables
private	Current contract only	Confidential data and helper functions	Functions & Variables

4. Detailed Explanation of Each Modifier

A. Public

1. **Functions:** Callable by *any entity* — internal or external.

```
2. function getBalance() public view returns(uint) { ... }
```

   a) Example: A mobile app or web interface can call this function directly.

3. **Variables:** Readable by anyone; Solidity auto-creates a *getter function.*

```
4. uint public totalSupply;  // Automatically creates totalSupply()
```

5. Be careful: anyone can read public variables and invoke public functions.

B. External

- **Accessible only from outside** the contract — not by internal functions directly.
- `function registerUser() external { ... }`
- Used for **API-style** functions called by other contracts or dApps.
- **Cannot** be invoked internally unless using `this.registerUser()`.

Property	Description
Purpose	Interface or API exposure
Cost	Slightly higher gas if called internally
Scope	External world only

C. Internal

- Accessible **within the same contract** and **any derived (inherited) contracts**.
- `function calculateReward() internal { ... }`
- Not visible to external users or apps.
- Used for **shared utility functions** or **internal data structures** between base and child contracts.

Think of `internal` as "protected" in OOP languages like Java or C++.

D. Private

- **Strictest modifier** — visible only within the same contract.
- `uint private secretCode;`
- `function _resetCounter() private { ... }`
- Not accessible by:

    a) External users
    b) Other contracts
    c) Derived contracts

Use Case	Example
Hide sensitive logic	internal validations, secret keys
Helper functions	small internal utilities

5. Inheritance Note (Preview)

1. Solidity supports **inheritance** — one contract can derive from another.
    a) A derived contract can access **internal** members of its base contract.
    b) But cannot access **private** members.

2. Example:

```
contract Base {
    uint internal baseCount;
    uint private baseSecret;
}

contract Derived is Base {
    // can access baseCount ✓
     // cannot access baseSecret ✗
 }
```

6. Summary Table

Modifier	Accessible by Same Contract	Accessible by Derived Contract	Accessible Externally	Typical Use
public	✓	✓	✓	General-purpose functions
external	✗ *(only via `this`)*	✗	✓	APIs and external calls
internal	✓	✓	✗	Shared internal logic
private	✓	✗	✗	Hidden helpers & sensitive data

7. Best Practices

1. Default to the **most restrictive** modifier necessary.
2. Avoid making variables **public** unless they need to be read externally.
3. Keep **private/internal** for core business logic.
4. Use **external** for interfaces or API functions that outside apps call.
5. Always comment visibility decisions for clarity and security audits.

8. Key Takeaways

1. Visibility modifiers control who can read, write, or invoke elements of a smart contract.
2. They ensure **security**, **data encapsulation**, and **API control**.
3. Mastering them is essential before learning inheritance and interface design.

Solidity Data Modifiers — Part 2

1. Introduction

In Solidity, apart from **visibility modifiers** (`public`, `private`, `internal`, `external`), there are **behavioral data modifiers** that define **how functions interact with the blockchain**.

They tell the **compiler** and **EVM** what the function does:

1. Whether it **reads or writes** blockchain state.
2. Whether it should **consume gas**.
3. Whether it should be treated as a **getter** or **setter** function.

2. Recap of Example Code Context

In the Solidity file **DataType.sol**, we have:

1. A **public** function `getStateVariables()` — a *getter* function.
2. A **private** function `setbMsg()` — a *setter* function.

These two illustrate how different **visibility** and **data modifiers** control function accessibility and blockchain interaction.

3. Public vs. Private Refresher

Function	Modifier	Meaning
`getStateVariables()`	`public`	Callable by anyone, returns stored values (getter).
`setbMsg()`	`private`	Only callable within the same contract (setter).

The **getter** retrieves blockchain-stored values (read-only).
The **setter** updates blockchain state (write operation).

4. Understanding the `view` Modifier

- `view` tells the compiler:

  "This function will **only read** from the blockchain and **not modify** any state variables."

Example:

```
function getStateVariables() public view returns (uint, string memory) {
    return (id, name);
}
```

- **Characteristics:**

  a) **No gas cost** when called *externally* (read-only).
  b) Cannot perform any **write** operations (e.g., modifying a state variable).

c) Compiler enforces this: any attempt to modify the blockchain will throw a **compiler error**.

Modifier	Reads Blockchain	Writes to Blockchain	Gas Used
`view`	✓ Yes	✗ No	✗ None (for external calls)

Best Practice:
Use `view` for read-only functions to reduce cost and prevent unintended changes.

5. The `pure` Modifier (Preview)

Though not used in this code yet, Solidity also allows the **`pure`** modifier.
It declares that:

The function **neither reads nor writes** from blockchain storage.

Example:

```
function add(uint a, uint b) public pure returns (uint) {
    return a + b;   // operates only on local variables
}
```

Modifier	Reads Blockchain	Writes Blockchain	Use Case
`pure`	✗ No	✗ No	Mathematical or local logic

6. State-Changing Functions

1. Functions **without `view` or `pure`** are *state-changing*.
2. They can **write to blockchain storage**, **update variables**, and thus **consume gas**.

Example:

```
function setbMsg(bytes32 newValue) private {
    bMsg = newValue;  // changes blockchain state
}
```

Modifier	Reads Blockchain	Writes Blockchain	Gas Used
(none)	✓	✓	✓ Yes

7. Getter and Setter Functions

Getter Function

- Used to **read** data from the blockchain.
- Typically marked `view` and `public`.
- No gas fee when called from a frontend or console.
- `function getStateVariables() public view returns(bytes32) {`

```
    return bMsg;
}
```

Setter Function

- Used to **update** data in the blockchain.
- Always **costs gas** since it changes blockchain state.
- Usually marked `public` or `private`.

```
function setbMsg(bytes32 newValue) private {
    bMsg = newValue;
}
```

Pattern:
`Public getter + Private setter = Safe encapsulation of blockchain data.`

8. Compiler Errors vs. Semantic Errors

1. **Compiler (Syntax) Errors:**
   a) Detected by the compiler before execution.
   b) Example: Typo, invalid syntax, or using blockchain access inside a `view` function.
   c) Helpful because they prevent faulty code from deployment.
2. **Semantic Errors:**
   a) Code compiles successfully but behaves incorrectly during execution.
   b) Example: Wrong logic or unintended behavior.
   c) Harder to detect—require **thorough testing**.

Type	When Detected	Example	Risk
Compiler Error	Before compilation	Using `view` but writing data	Low
Semantic Error	After deployment	Wrong logic or variable misuse	High

✅Compiler errors are *good* — they protect you from costly blockchain mistakes.
❌Semantic errors are *dangerous* — they may corrupt logic in deployed contracts.

9. Returning Multiple Values

1. Solidity functions can **return multiple values** simultaneously.

```
2. function getStateVariables() public view returns (uint, string memory)
   {
3.     return (x, message);
4. }
```

5. Enables concise data retrieval from the blockchain.

10. Summary Table

Modifier	Function Type	Reads State	Modifies State	Gas Used	Typical Use
view	Read-only	✓	✗	✗	Display or fetch stored data
pure	Local computation	✗	✗	✗	Calculations or validations
(none)	State-changing	✓	✓	✓	Update contract data
private / public	Visibility control	depends	depends	depends	Restrict or allow access

11. Best Practices

✓Use `view` for all **getter** functions.
✓Use `private` for **setter** functions to protect internal data.
✓Always handle compiler warnings and red squiggles before deployment.
✓Minimize state updates — each change costs **gas**.
✓Test logic carefully to avoid **semantic errors** in deployed contracts.

12. Key Takeaways

- `view` → reads only, no gas.
- `pure` → uses only local data, no blockchain access.
- No modifier → modifies state, consumes gas.
- `public`/`private` → define who can call these functions.
- Compiler errors are friends; semantic errors are silent enemies.

Revisiting Gas in Solidity

1. Introduction

1. Every interaction with the **Ethereum blockchain** requires **gas**.
2. Gas represents the **computational energy** needed to execute operations and record them permanently on the blockchain.
3. Gas ensures the network remains **secure, efficient, and spam-resistant**.
4. **Analogy:**
   Think of gas as **fuel for a car** — you need it to move (execute), and you pay based on how far or how complex your trip is.

2. Why Gas Matters

1. Gas ensures that **miners (validators)** are compensated for executing and validating transactions.
2. It also prevents **infinite loops or wasteful code** from clogging the network.
3. Developers must manage gas efficiently to avoid excessive transaction costs.

3. Key Gas Concepts

Solidity and Ethereum use **three core gas parameters** that determine transaction cost and execution behavior:

Term	Meaning	Analogy
Gas Price	The amount of Ether (ETH) per unit of gas you're willing to pay.	Price per litre of fuel
Gas Limit	The maximum number of gas units you're willing to spend for the transaction.	Size of your fuel tank
Gas Cost	The actual gas consumed by each operation (opcode) in your contract.	Fuel consumed per kilometer

4. 1 Gas Price

1. Defined by the **transaction originator** (sender) in the transaction header.
2. Represents the **maximum price per gas unit** the user is willing to pay (in wei).
3. Determines **how attractive your transaction is** to miners.

Mechanism:

- **Higher gas price** → Faster inclusion in a block (miners prioritize it).
- **Lower gas price** → Slower confirmation time.

⛽ **Analogy:**
You're willing to pay ₹120 per litre (high gas price), so petrol stations (miners) prioritize serving you before others paying ₹90 per litre.

5. 2 Gas Limit

1. Specifies the **maximum number of gas units** a user allows for the transaction.
2. Each operation (e.g., addition, multiplication, storage update) consumes a **fixed amount of gas**.
3. The **gas limit** should be sufficient to complete the entire transaction.
4. **If gas limit is too low:**

1. The transaction **fails and reverts**.
2. The unspent portion of gas is returned, but the spent part for execution attempts is **lost**.
3. This can happen if the function is more complex than expected.

If gas limit is too high:

1. Transaction succeeds, and **unused gas is refunded** to the sender.

Case	Outcome
Gas limit < required gas	Transaction fails (out of gas)
Gas limit > required gas	Transaction succeeds; unused gas refunded

Infinite Loop Protection:
If your code accidentally loops infinitely, it will **run out of gas** and stop automatically — preventing the network from hanging.

6. 3 Gas Cost

1. Every **operation** in Solidity has a **fixed cost** in gas units.
2. Examples:

Operation	Gas Units
`ADD` **(addition)**	3
`MUL` **(multiplication)**	5
`SSTORE` **(store variable)**	20,000+

3. Complex functions that use many operations or modify blockchain storage consume more gas.

Rule of Thumb:
Optimize your algorithms to **minimize total gas consumption** — less computation, fewer state updates.

7. Transaction Fee

- The **total transaction fee** you pay =

  Total Gas Used×Gas Price

- This total fee compensates miners for executing and recording your transaction.

Term	Formula / Example
Gas Used	Total units consumed (e.g., 50,000 gas)
Gas Price	20 gwei (1 gwei = 10^{-9} ETH)
Transaction Fee	50,000 × 20 gwei = 1,000,000 gwei = 0.001 ETH

Note:
The transaction originator always pays the fee in **Ether**, even if interacting with a token or contract.

8. Unused Gas

- After execution, any **unused gas** (from the difference between gas limit and gas consumed) is **refunded** to the sender's account.
- This ensures fairness — users don't pay for unused computation.

Example:

- You set `gasLimit = 100,000` and use only `85,000`.
- The remaining `15,000` gas units' worth of ETH is **returned** to your wallet.

Analogy:
You budget ₹100 for petrol to drive to the beach. The trip only costs ₹85 — the leftover ₹15 goes back into your pocket.

9. Summary of Gas Elements

Parameter	Description	Who Defines It	Impact
Gas Price	ETH per unit of gas	Transaction sender	Determines miner priority
Gas Limit	Max gas allowed	Transaction sender	Sets execution boundary
Gas Cost	Gas used per operation	Fixed by EVM	Determines complexity
Transaction Fee	Total cost (Gas Used × Gas Price)	Calculated	Paid in Ether
Unused Gas	Returned to sender	By EVM	Saves user balance

10. Best Practices to Optimize Gas

✅Minimize blockchain **state updates** (e.g., use memory variables over storage when possible).
✅Combine operations to reduce computational steps.

✅Use **view** and **pure** modifiers for read-only functions — they **don't cost gas** when called externally.
✅Avoid **infinite loops** or **nested iterations** over large datasets.
✅Use **smaller data types** (`uint8`, `uint16`) only if you truly need small ranges.

11. Moral of the Story

- You **pay gas for every blockchain interaction**.
- Understand **Gas Price**, **Gas Limit**, and **Gas Cost** to design efficient smart contracts.
- Write clean, optimized code that saves your users' Ether and ensures transactions succeed.

⚡ **Final Analogy:**
Blockchain is a global highway.

- **Gas = fuel**
- **Gas price = how much you pay per litre**
- **Gas limit = size of your fuel tank**
- **Unused gas = leftover fuel returned to you**
  The goal: **Reach your destination (execute your transaction)** efficiently — without running out of gas!

Controlling Flow in Solidity

1. Introduction

- So far, our Solidity examples were simple: define data, create functions, return values.
- Real-world contracts require **decision-making** and **repetition** — to react to conditions and process multiple items.
- Solidity provides **flow control structures** that help us guide **program execution** based on logical conditions.

Flow Control Type	Purpose
Conditionals	Execute code selectively (e.g., `if`, `else`)
Loops / Iterations	Repeat code until a condition changes (e.g., `while`, `do-while`, `for`)

2. Smart Contract Example

A simple contract named `flowControl` contains **four functions** demonstrating:

1. `calcDozenDiscountIF()` → *if–else statement*
2. `calcDozenDiscountWHILE()` → *while loop*

3. `calcDozenDiscountDO()` → *do–while loop*
4. `calcDozenDiscountFOR()` → *for loop*

Each function helps calculate a **"dozen discount"** for a donut purchase — an intuitive, business-like example for understanding control flow.

3. The `if-else` Statement

Used for **conditional decision-making**.

Example Function:

```
function calcDozenDiscountIF(uint purchasedQty) private pure returns(bool) {
    bool giveDozenPrice = false;  // Always initialize variables!
    if (purchasedQty >= 12)
        giveDozenPrice = true;
    else
        giveDozenPrice = false;
    return giveDozenPrice;
}
```

Explanation:

- Checks if `purchasedQty ≥ 12`.
- If true → `giveDozenPrice = true` (eligible for discount).
- If false → remains `false`.
- Returns a boolean indicating discount eligibility.

Key Concept:

- `if` block → runs when condition is `true`.
- `else` block → runs when condition is `false`.
- Curly braces `{}` group multiple statements inside one condition.

Best Practice: Always **initialize variables** to prevent undefined states.

---

4. The `while` Loop

Used for **repeated execution** as long as a condition is true.

Example Function:

```
function calcDozenDiscountWHILE(uint purchasedQty) private pure returns(bool)
{
    bool giveDozenPrice = false;
    uint numDonuts = 1;
```

```
    while (numDonuts < purchasedQty) {
        numDonuts++;
        if (numDonuts >= 12) {
            giveDozenPrice = true;
            break; // exit the loop
        }
    }
    return giveDozenPrice;
}
```

Explanation:

- Starts with `numDonuts = 1`.
- Keeps iterating while `numDonuts < purchasedQty`.
- If count reaches 12 → sets discount to true and uses **`break`** to exit the loop.
- Otherwise, continues iterating.

Feature	Description
Entry Test	Condition checked **before** executing loop body.
Execution	0 or more times (may not run at all).
Exit	Triggered when condition becomes false or `break` used.

In Solidity: Loops are bounded by **gas limits** — infinite loops will automatically fail once gas runs out.

5. The `do-while` Loop

Used for **guaranteed one-time execution**, even if condition is false initially.

Example Function:

```
function calcDozenDiscountDO(uint purchasedQty) private pure returns(bool) {
    bool giveDozenPrice = false;
    uint numDonuts = 1;

    do {
        numDonuts++;
        if (numDonuts >= 12) {
            giveDozenPrice = true;
            break;
        }
    } while (numDonuts < purchasedQty);

    return giveDozenPrice;
}
```

Key Difference:

Loop Type	Condition Check	Minimum Executions
`while`	Before body	0
`do-while`	After body	1

Analogy:

- `while`: Check ticket **before entering** the concert.
- `do-while`: You're **allowed inside once**, and then your ticket is checked to decide if you can stay.

6. The `for` Loop

Compact, structured loop that embeds **initialization**, **condition**, and **increment** in one line.

Example Function:

```
function calcDozenDiscountFOR(uint purchasedQty) private pure returns(bool) {
    bool giveDozenPrice = false;

    for (uint numDonuts = 1; numDonuts <= purchasedQty; numDonuts++) {
        if (numDonuts >= 12) {
            giveDozenPrice = true;
            break;
        }
    }
    return giveDozenPrice;
}
```

Explanation:

- Initialization: `numDonuts = 1`
- Condition: `numDonuts <= purchasedQty`
- Increment: `numDonuts++` after each iteration.
- Sets discount true when 12 or more donuts are reached.

✅**Advantages:**

- Clean, compact syntax.
- All loop control variables in one place.
- Ideal for fixed-range iterations.

7. Flow Control Summary

Structure	Purpose	Runs When	Execution Count	Solidity Example

`if-else`	Conditional branch	Condition true/false	1 time	Discount check
`while`	Loop until condition fails	Condition true	0–∞	Continuous checking
`do-while`	Loop at least once	Condition tested after body	1–∞	Guaranteed one run
`for`	Indexed iteration	Condition true	Controlled by index	Counter-based

---

8. Key Solidity Considerations

Initialization: Always assign initial values to avoid unpredictable behavior.
Local Variable Scope: Local variables with the same name in different functions don't conflict.
Break Statements: Use `break` to safely exit loops when a condition is met.
Gas Usage: Avoid unnecessary loops or heavy computations — they consume **more gas**.
Use Case: Flow control is crucial for conditional pricing, validation, and automated contract logic.

9. Practical Analogy

Imagine an **online donut shop** smart contract:

- **`if-else`** → Check if the user qualifies for a bulk discount.
- **`while`** → Process each donut order until the desired quantity is reached.
- **`do-while`** → Process at least one order even if user entered zero.
- **`for`** → Automatically iterate through fixed donut counts or batches.

10. Best Practices

✅Keep loops **short and finite** — long or infinite loops may exceed gas limits.
✅Use **`if-else`** for decision-making; prefer **`for`** for deterministic loops.
✅Always **initialize** local variables before use.
✅Avoid heavy on-chain computations — use **off-chain pre-calculation** if possible.
✅Add comments and organize curly braces `{}` for readability.

11. Conclusion

- Solidity supports standard **flow control mechanisms** similar to traditional programming languages.

- These structures allow smart contracts to **react, iterate, and decide** intelligently.
- Mastering flow control is essential for building robust dApps and supply-chain–style business logic.

Handling Errors in Solidity

1. Introduction

Even the best-written smart contracts encounter **unexpected conditions** — faulty inputs, invalid users, or unforeseen states.
In Solidity, **error handling** ensures:

- Contract stability
- Protection of blockchain data integrity
- Proper communication of failure reasons

Key Idea:
"Plan for failure."
Preventing and responding to errors is just as critical as writing logic that works.

2. Evolution of Error Handling in Solidity

- **Early versions** of Solidity used the `throw` statement to terminate a transaction.
- `throw` **is now deprecated** (no longer used).
- Modern Solidity uses three mechanisms:
    1. `require()` – **Prevent** errors before they happen
    2. `revert()` – **Gracefully exit** when something goes wrong
    3. `assert()` – **Critical stop** for unexpected, fatal errors

3. The `require()` Function

Used for **precondition checks** — verifies that certain conditions are true **before executing** code.

Purpose

- Prevents invalid function calls.
- Automatically **reverts** state changes if the condition fails.
- **Refunds unused gas** to the sender.
- Optionally provides an **error message** for debugging.

Syntax

```
require(condition, "Error message");
```

Example

```
function withdraw(uint amount) public {
    require(msg.sender == owner, "Not the contract owner");
    require(amount <= balance, "Insufficient funds");
    balance -= amount;
}
```

Aspect	Description
When used	Before running risky or privileged code
Effect	Stops execution if condition fails
Gas	Unused gas is refunded
Use Case	Access control, input validation, ownership check

 Analogy:
Check ID *before* pouring beer. If not valid → don't start the process at all.

4. The `revert()` Function

Used to **manually stop** execution when an expected but undesired condition occurs.

Purpose

- Roll back all state changes within the current transaction.
- Return an optional **error message**.
- **Refund unused gas** to the transaction sender.
- Useful when conditions are checked *after* partial processing.

Syntax

```
revert("Reason for failure");
```

Example

```
function transfer(address recipient, uint amount) public {
    if (msg.sender != owner) {
        revert("Only owner can transfer");
    }
    // Continue transfer if condition passes
}
```

Aspect	Description
When used	When something unexpected but recoverable happens
Effect	Reverses all state changes
Gas	Unused gas refunded
Use Case	Safe exit after validation fails mid-function

Analogy:
You start the car but notice a problem → stop and reverse immediately, returning fuel for the untraveled distance.

5. The `assert()` Function

Used for **internal sanity checks** — to detect conditions that should *never* occur.

Purpose

- Detect *programming bugs* or *impossible logic states*.
- **Consumes all remaining gas** (no refund).
- Should be used **rarely** and only for debugging or enforcing contract invariants.

Syntax

```
assert(condition);
```

Example

```
function safeMath(uint a, uint b) internal pure returns (uint) {
    uint c = a + b;
    assert(c >= a);  // If this fails, something is fundamentally broken
    return c;
}
```

Aspect	Description
When used	Detect impossible or critical logical errors
Effect	Halts execution and **burns all gas**
Use Case	Testing invariants, catching fatal conditions

Important:
Avoid using `assert()` for input validation — use `require()` instead.
`assert()` = developer check, not user check.

6. Behavior Comparison

Function	Purpose	Refunds Gas?	Typical Use	Example Scenario
`require()`	Prevent error *before* executing code	✓ Yes	Input validation / access control	`require(msg.sender == owner)`
`revert()`	Graceful exit *after* error detected	✓ Yes	Controlled rollback	`revert("Transfer failed")`
`assert()`	Critical failure (bug)	✗ No	Internal checks / logic validation	`assert(totalSupply >= 0)`

7. How They Handle State & Gas

Function	State Changes	Remaining Gas	Refund Behavior
`require()`	Rolled back	Returned	Refunds unused gas
`revert()`	Rolled back	Returned	Refunds unused gas
`assert()`	Rolled back	Consumed	No refund

All three **undo state changes** made within the current call — preserving blockchain consistency.

8. Error Handling Strategy

Step	Strategy	Function
1	**Prevent errors early**	Use `require()`
2	**Gracefully exit known issues**	Use `revert()`
3	**Handle impossible failures**	Use `assert()`

✅Best Practice Flow:

```
require(validCondition, "Pre-check failed");
// perform logic
if (unexpectedCase) revert("Unexpected issue");
assert(alwaysTrueCondition);
```

9. Gas and Error Handling

- **Failed transactions still consume gas** up to the failure point.
- **require()** and **revert()** refund unused gas, minimizing cost.
- **assert()** uses all gas → avoid except in debug or invariant testing.

Plan error handling efficiently to save **gas and Ether** for users.

10. Deprecated: `throw`

- In early Solidity versions, errors were handled using `throw`.
- **Now deprecated** — replaced by safer, more informative methods.
- Never use `throw` in new contracts.

11. Best Practices

✅Always use `require()` for **input validation and access control**.
✅Use `revert()` to **gracefully undo** a partially completed function.
✅Use `assert()` sparingly — only for detecting **developer mistakes**.
✅Include **meaningful error messages** for transparency and debugging.
✅Test error-handling scenarios in your unit tests using frameworks like **Truffle** or **Hardhat**.

12. Example Summary

```
function changeOwner(address newOwner) public {
    require(msg.sender == owner, "Only current owner can change owner");
    if (newOwner == address(0)) {
        revert("Invalid address");
    }
    assert(owner != newOwner); // sanity check
    owner = newOwner;
}
```

- `require()` → pre-check access
- `revert()` → rollback on invalid input
- `assert()` → internal check to catch impossible condition

13. Key Takeaways

- Solidity provides structured, gas-efficient error handling.
- Always **prevent → handle → safeguard** in that order:
    1. **Prevent** errors using `require()`
    2. **Handle** known issues using `revert()`
    3. **Safeguard** against bugs using `assert()`
- This ensures both **code reliability** and **user protection**.